

A vertical brown line is positioned to the left of the title. The background features a light gray vertical band on the left, a white area on the right, and three solid brown rectangular blocks: one in the top right, one in the bottom left, and one in the bottom right.

Course Registration Project Proposal

Aarav Mehta

Current System Issues

Performance

Severe latency and lag during peak usage periods (e.g. first week of the semester)

User Interface

Outdated UI is frustrating to students, leading to registration errors and confusion.

Management

Fragile, decades-old code is difficult to maintain, build on, or manage for more users.

Integration

Lack of modern framework usage prevents us from improving the platform as new technology comes out.

Overview:

Our new system will:

- Use load balancing to handle traffic over a large server network.
- Dynamically scale maximum capacity for the start of semesters.
- Implement a five-stage course registration system to handle concurrent enrollment and reduce the negative impact of high latency.
- Use asynchronous processing to minimize latency and make enrollment as fair as possible for all students.

Assumptions:

- 40,000 total students
- 5,000 faculty members
- 2,000 course sections per semester
- Heavy traffic skew
- Most users will be accessing from the university.
- Peak load: First min. of registration
 - 1/5 students sending requests
 - 10–20 requests/course over 30 seconds
 - 0.5 req/second/user avg.
 - 4,000 req/second maximum
- Data storage for 40,000 users, entirely textual, negligible size.
- University system handles student auth.
- Enough labs to accommodate lectures

Functional Requirements

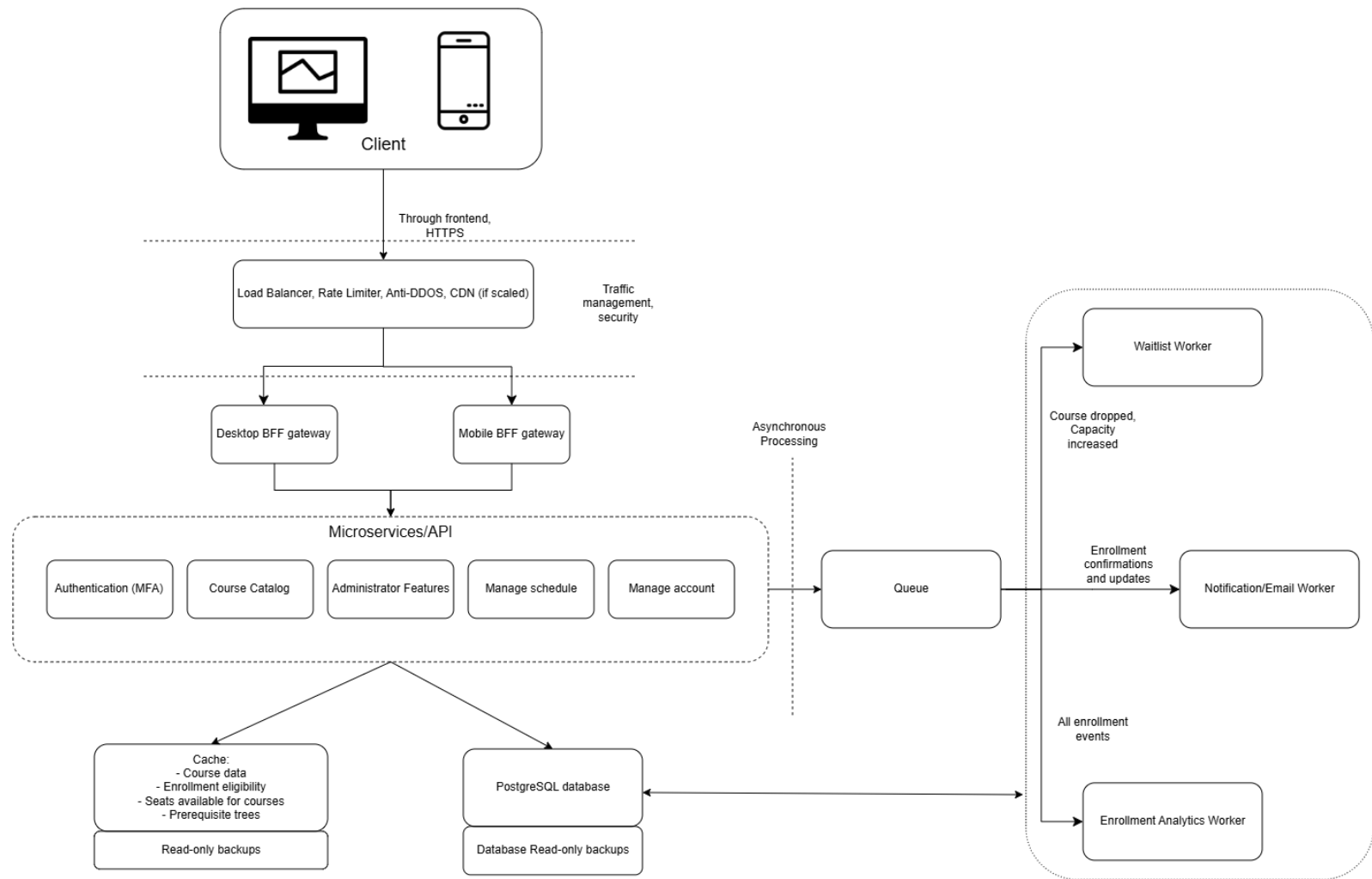
- Student login linked through university email verification system
- Robust course search system
 - By course name, code, professor, time, day
 - Relevance, then popularity
- View course details, information, registrations
- Register/Drop/Waitlist courses with appropriate restrictions
- Personal schedule creator
 - UX is key
- View semesters, future semester offerings
- Notification system
- Integrated degree progress view

Administrators only:

- Manage courses, listings, and semesters
- Generate course bypass codes to provide to professors
- View all student enrollments and rosters

Non-functional Requirements

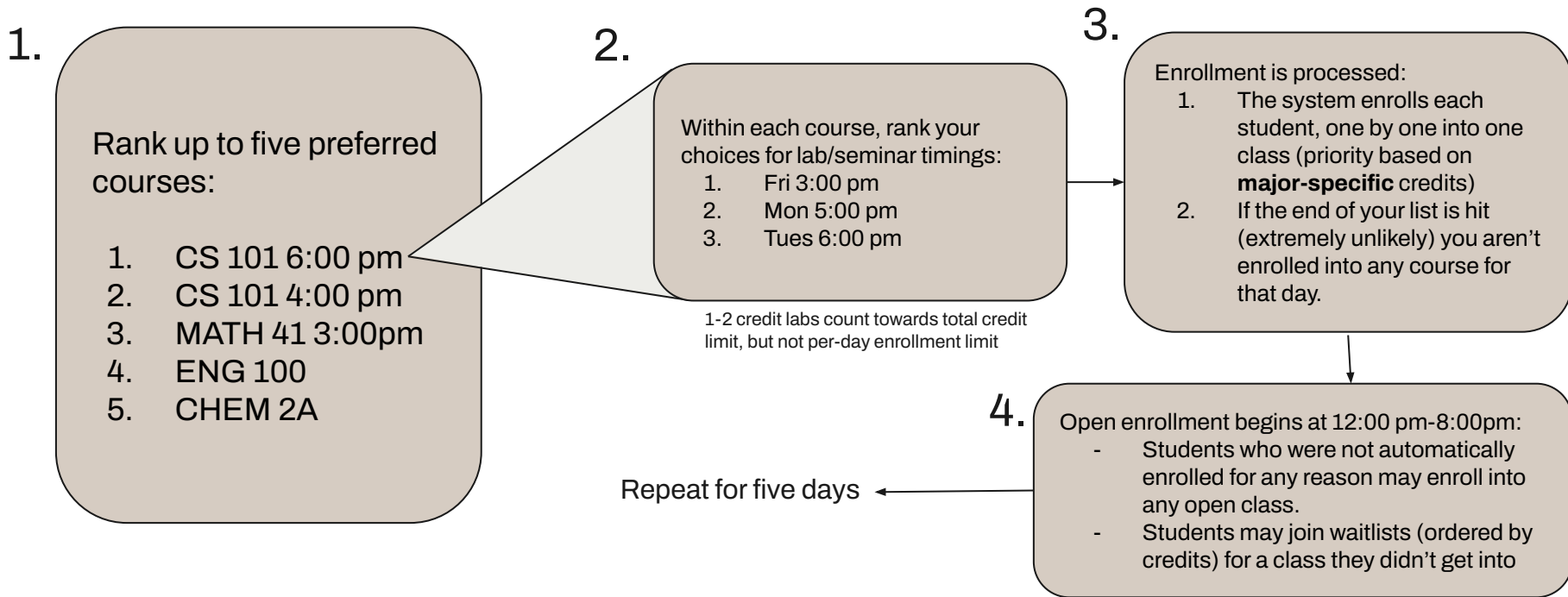
- Availability: needs to be accessible without failure at all times.
 - If part of the system critically fails, the whole system must shut down.
- Performance: low latency and constant updates on the client-side.
- Reliability: Minimal issues preventing students from registering, safeguards against potential problematic version changes/updates.
- Scalability: Potential to expand to multiple universities efficiently without sacrificing performance.
- Security: Encryption of all sensitive student information exchanged between server and client, rate-limiting to prevent enrollment bots.
- Maintenance: Well-organized and independent backend component microservices that can be checked for regular upkeep.



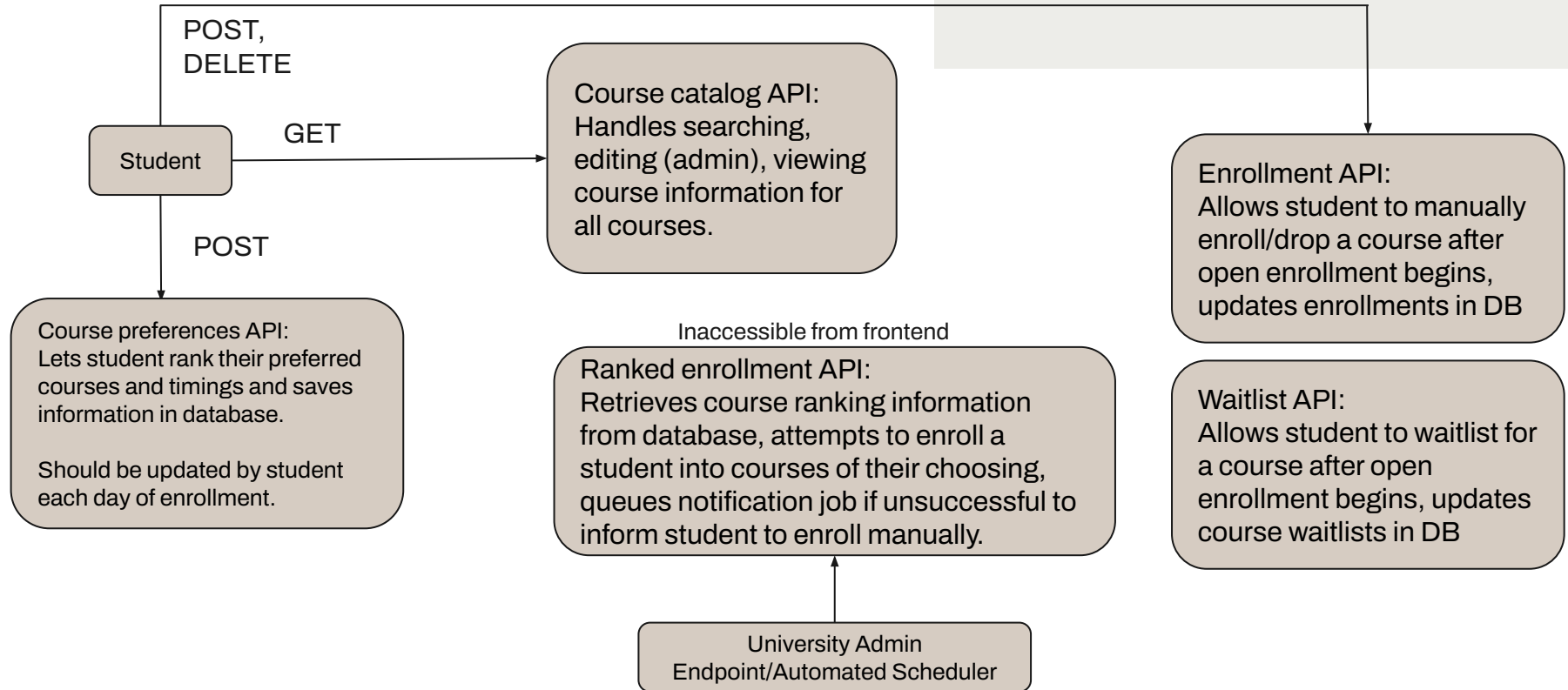
Course Registration: Deep Dive

Five day wave-based system:

- 4–6 classes per student
- A standard full time class is 5 credits
- On course registration week:
 - Each day Mon–Fri, students register for one course.



Registration API/backend breakdown



Database Design

Tables:

- Students: **SID**, Name, email
- Instructor: **IID**, Name, email
- Student Rankings: **PID**, *SID*, Term, Day, Preferences (array of json objects defined as a typescript interface), Timestamp
- Enrollment: **EID**, *SID*, *CSID*, day
- Course: **CID**, course title, credits, prerequisites (boolean tree of CID)
- Course Section: **CSID**, *CID*, start time, end time, days/week, *IID*, capacity, term
- Waitlist: **WID**, *CSID*, *SID*, position
- Student Course History: **HID**, *SID*, *CID*, term, grade, credits
- Bypass Codes: **ID**, *CSID*, expiry

Duplicates for
lab sections

Indexes:

- Enrollment: (SID, day) to ensure maximum one enrollment per day
- Course History: (SID, CID, credits) to verify prerequisites for a student
- Waitlist: position to easily find the next person to move up on the waitlist

Constraints:

- No duplicate course enrollments
- One enrollment only per day
- One course ranking per day
- No duplicate waitlist positions

API Design

Course Catalog (/api/v1/courses):

- GET: searches courses by ID, name, department
- GET (/courses/CID): shows information for one specific course

Course Ranking (/api/v1/courseranking):

- PUT: Update a student's preferred rankings for each day
 - Payload: course ranking JSON
- GET: displays saved preference information on page load

Enrollment (/api/v1/enroll):

- POST: Enroll student into an open section manually if they have a remaining enrollment for that day
 - Payload: Intended course and term enrollment info
- DELETE: Drop an existing enrollment
- GET: displays saved enrollment information on page load

Waitlist (/api/v1/waitlists):

- POST: Enroll student into the waitlist for a course.
 - Payload: Intended course and term enrollment info
- DELETE: Remove student from the waitlist and update positions.
- GET: displays saved waitlist information on page load

API Design cont.

Auto-enrollment (/api/v1/admin/auto-enroll):

- POST: executes the automatic enrollment based on student rankings
 - Payload: enrollment day # and term

Schedule (/api/v1/schedule):

- GET: Retrieves and displays student's current course schedule as a list and a calendar.
- POST: Adds an available university course to the schedule view to assist with planning
 - Payload: Course ID

Prof. Course Management (/api/v1/instructor/courses):

- GET: displays all courses a professor is in charge of
- GET(/courses/CID): displays information for one course
- PUT: edits or adds information for a course
 - Payload: Course information
- POST: creates a course bypass code for students
 - Payload: Course information

Admin metrics (/api/v1/admin/metrics):

- GET: Provides live overview of system traffic and tasks for the current day
- GET (/metrics/day/#): Provides system metrics history for a previous course enrollment day.

Reliability

Potential Failures:

- Database unavailable:
 - If backups are healthy, attempt to use one as the primary database, otherwise switch to read-only mode,
- Cache unavailable:
 - Swap to a backup if healthy, otherwise swap to direct database queries with write-request throttling to prevent the database from overloading.
- Duplicate registration requests:
 - Check for uniqueness in enrollments table, block all but the first copy of a registration.
 - One course per day system makes this issue easy to solve.
- Network failure:
 - Roll back enrollment in transactions if not entirely finished, delay wave-based registration schedule as needed until network returns.
- Worker failures:
 - Notification: Queue notifications to send when worker is back up.
 - Waitlist: Dropped seats are reserved for the waitlist worker to fill when it comes back up.

Scalability

Starting Capacity: 40,000 students

- Convenient: most students log on from the same location.
- Unified storage for a single university.
- Fewer network delays.

Small University System: 100,000 students

- Multiple servers dedicated to university locations.
- Same basic system infrastructure, but for multiple universities.
 - Split database tables by campus, introduce campus_id.
- Bottleneck: CPU
 - Different enrollment times/systems can help alleviate this.

Large University System: 500,000 students

- Multiple backend systems dedicated to processing 1-2 universities each.
- Asynchronous processing for registration and enrollment so each university can operate this independently.
- Bottleneck: CPU and potentially caching
 - Redis could struggle to load large prerequisite and ranking trees/lists for thousands of unique courses.

Nationwide platform: >1M students

- Split universities into regional or state groups and build separate infrastructure to handle each group.
 - One system going down shouldn't have any impact on the others.
- Cache important assets in CDNs near major usage sites.
- Bottleneck: CPU and caching
 - Need separate systems and databases scaled to handle the large amount of requests and information.

Security and Monitoring

- Authentication:
 - Redirect to university auth SSO, wait for an OK response, store JWT for 30 minutes after.
- Authorization:
 - Student, Professor, Administrator
 - /instructor endpoint restricted against students
 - /admin endpoint restricted to administrator only
- Data transfer
 - All data sent over the network is on HTTPS, encrypted with TLS
- Rate limiting
 - Traffic is managed to prevent spamming from the same IP address.
 - Rate limit is further reduced during enrollment week days when service is most active.
- Audit Logs
 - Actor, action, and all relevant data is stored in a log with a timestamp
 - Audit logs stored in a separate read-only system to prevent tampering.

Metrics:

- Logs from console implemented in backend programming/processing
 - Important slow processing/warning/error logs are saved and displayed on metrics dashboard
- Average query time
- Maximum query time
- Average RPS, peak RPS within last minute
- # of write requests to database within last min
 - Read and write frequency differ heavily in this kind of system
- Notification and waitlist workers
 - Tasks in both queues
 - Courses with >100 students on one waitlist displayed on dashboard
- Graph display of RPS over time

Tradeoffs

1. Five day enrollment waves with one course per day

Pros:

- All students have the fairest chance at enrolling into their preferred classes.
- Organized and simple system that doesn't require any inconvenient enrollment appointment times.
- Easier to prevent botting and abuse.

Cons:

- Users must put effort in to create a ranked list.
- More expensive in terms of storage and processing (long trees and lists).

2. Capping user rank choices to 5

Pros:

- Easily predictable database storage and json objects.
 - Improved metrics
- Lower runtime during automatic enrollment.

Cons:

- Less student freedom in course ranking.
- Small chance of all five courses the user lists being full and not getting a class.

Looking to the Future

AI Schedule Planning

Smart schedule optimization and automated degree planning for students.



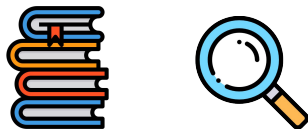
Advanced Analytics

Statistics-based prediction algorithms and institution-specific information.



Course Curriculum in Catalog

Direct integration of a course syllabus and curriculum information in course information.



LMS Integration

Third-party support for major LMS platforms.

